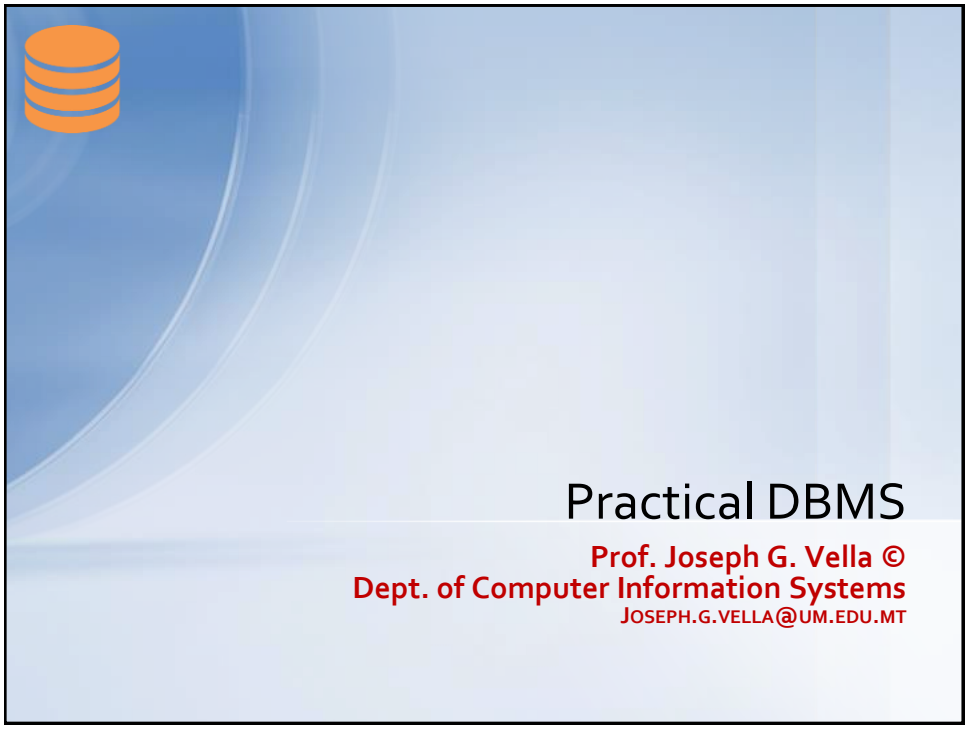

Practical DBMS
Prof. Joseph G. Vella ©
Dept. of Computer Information Systems
JOSEPH.G.VELLA@UM.EDU.MT

1


Practical DBMS
Rationale & Challenges

2

Where had we started?

- **Definition (first cut):**
  A **database** is a collection of structured data stored on persistent storage.
  This starting definition is apt for the Relational Model …

- **Environment and Contextual (very real):**

  Available to a large number of end users.
  - **As in "Access my data when I need it …"**
    - **No delay, no access denial, no loss of data.**

  Correctness of computations and upholding assertions.
  - **As in "What's my balance?" gets the right value until it is changed.**

3

How is usage managed and controlled?

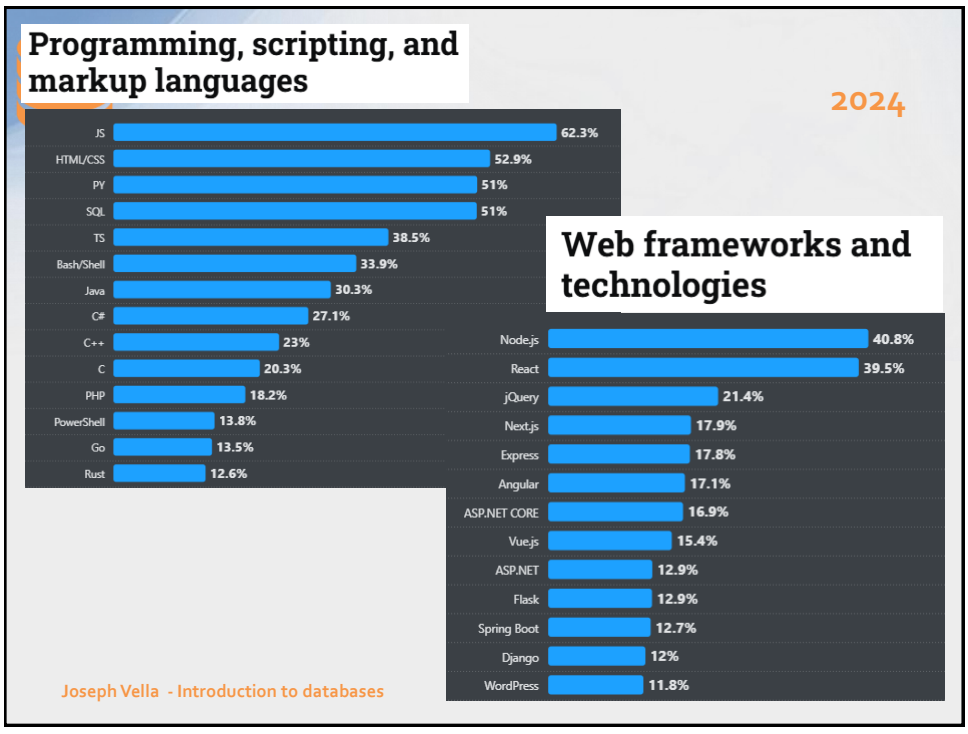
- **A general-purpose Database Management System (DBMS)**
  Is a computer-based application tool set for:
  defining, creating, manipulating, controlling, and managing ***databases***.

- **Requirements demanded on a DBMS:**
  End users, developers, administrators demand
  flexibility & generality
  in terms of allocation of computational resources and storage space. Also, the following feature highly on requirements list:
  - **Reliability** (e.g., in terms of resilience to faults and interruptions)
  - **Openness** (e.g., in terms of data connectivity)
  - **Scalability** (e.g., in terms of storage and throughput)

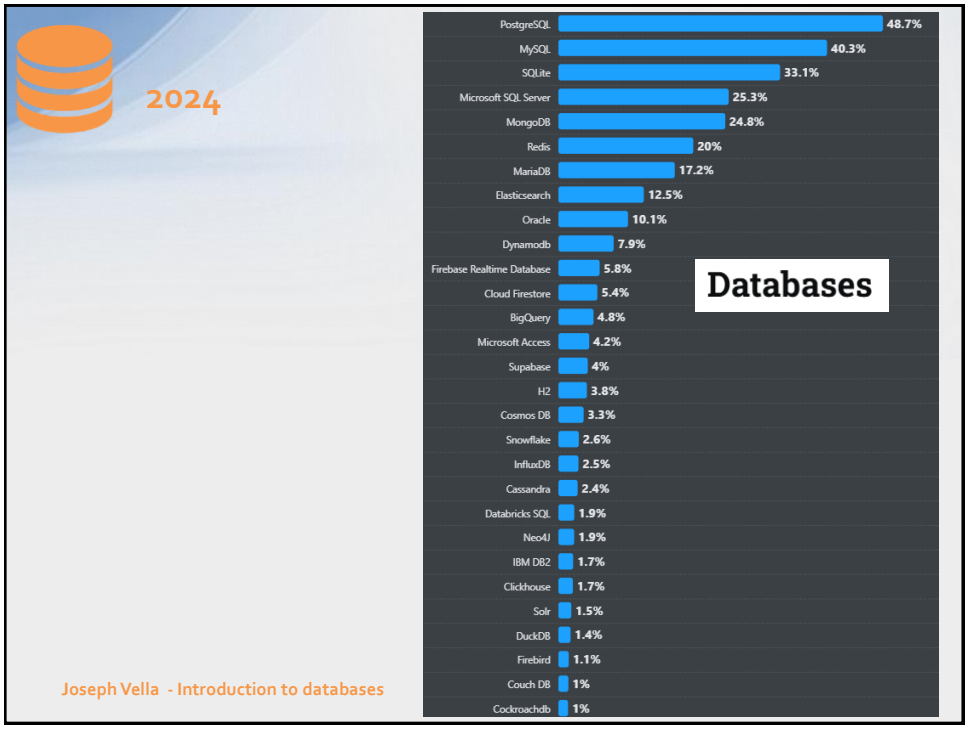
4

Available DBMS (a biased collective list)
Stackoverflow 2024 ()

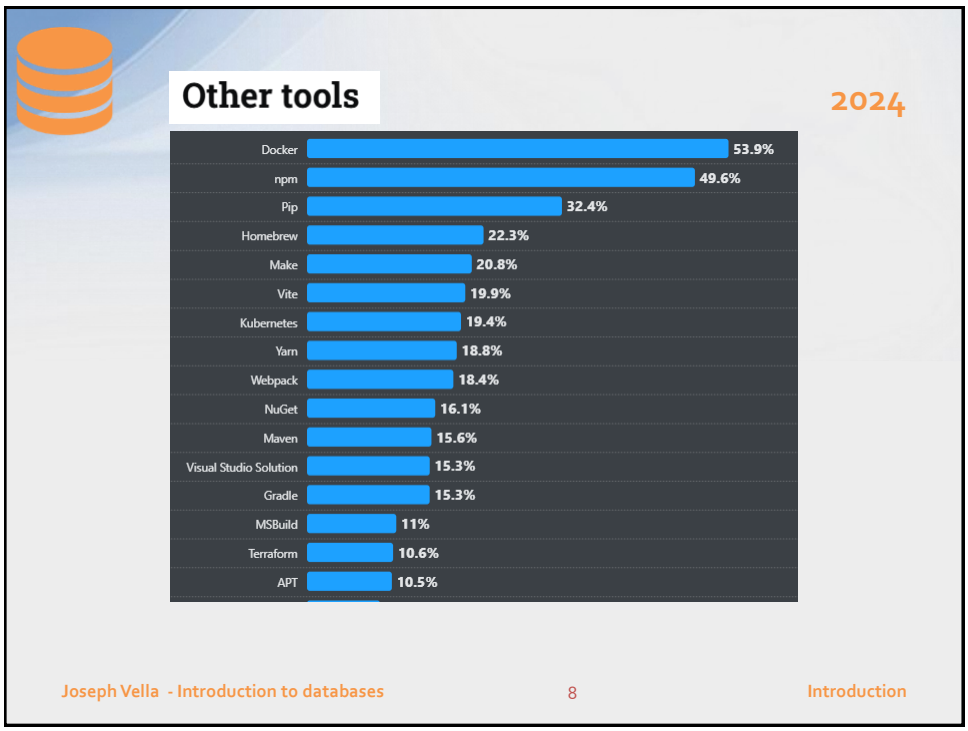
5



6


2024
Databases
PostgreSQL 48.7%
MySQL 40.3%
SQLite 33.1%
Microsoft SQL Server 25.3%
MongoDB 24.8%
Redis 20%
MariaDB 17.2%
Elasticsearch 12.5%
Oracle 10.1%
Dynamodb 7.9%
Firebase Realtime Database 5.8%
Cloud Firestore 5.4%
BigQuery 4.8%
Microsoft Access 4.2%
Supabase 4%
H2 3.8%
Cosmos DB 3.3%
Snowflake 2.6%
InfluxDB 2.5%
Cassandra 2.4%
Databricks SQL 1.9%
Neo4J 1.9%
IBM DB2 1.7%
Clickhouse 1.7%
Solr 1.5%
DuckDB 1.4%
Firebird 1.1%
Couch DB 1%
Cockroachdb 1%
Joseph Vella - Introduction to databases

7

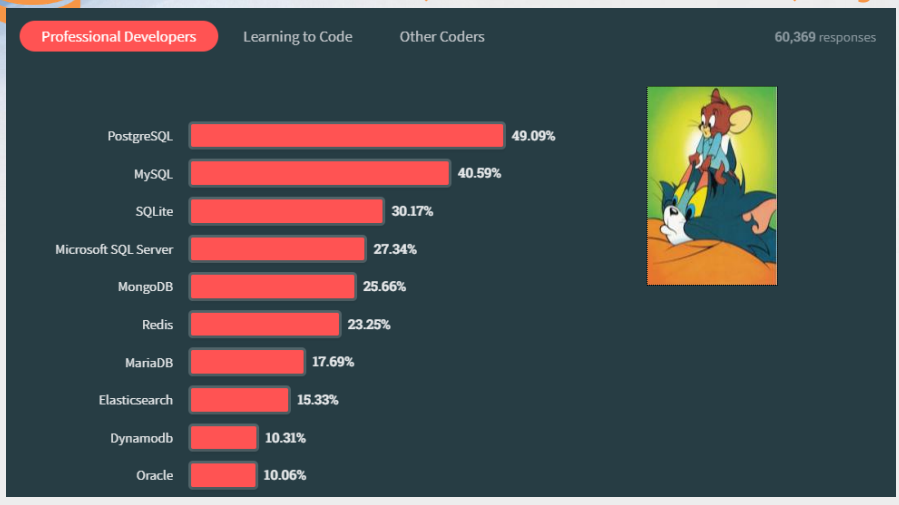

Other tools
2024
Docker 53.9%
npm 49.6%
Pip 32.4%
Homebrew 22.3%
Make 20.8%
Vite 19.9%
Kubernetes 19.4%
Yarn 18.8%
Webpack 18.4%
NuGet 16.1%
Maven 15.6%
Visual Studio Solution 15.3%
Gradle 15.3%
MSBuild 11%
Terraform 10.6%
APT 10.5%
Joseph Vella - Introduction to databases
8
Introduction

8

Available DBMS (a biased collective list)
Stackoverflow 2023 (May)

9

Eye opener!?
DBMS (and other fields of interest) 2023



- **Look out … this numbers are personal.**
  i.e., Not reflecting data requirements.

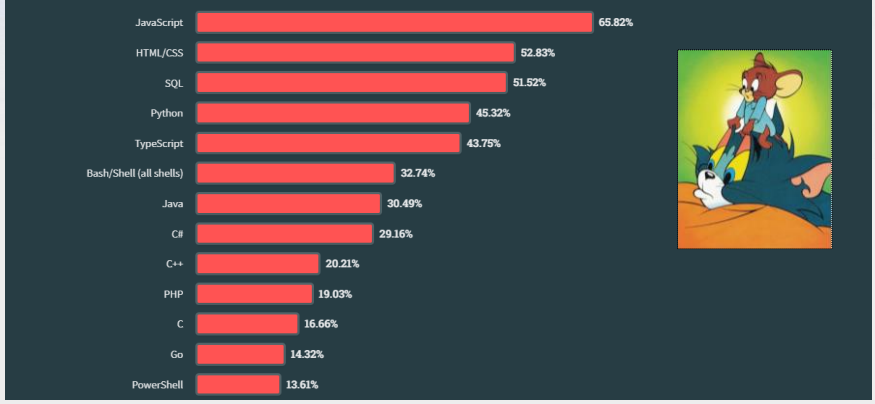
10 J Vella (c) - PDBMS Introduction

10

Eye opener!?
DBMS (and other fields of interest) 2023

Professional Developers | Learning to Code | Other Coders

67,053 responses

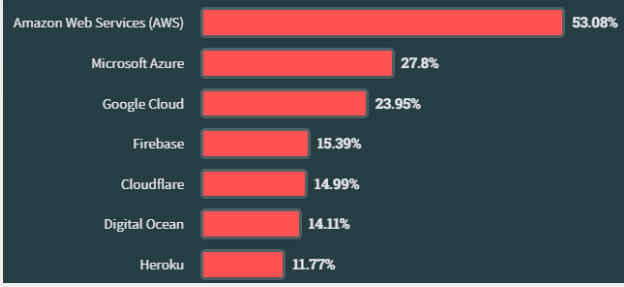


11

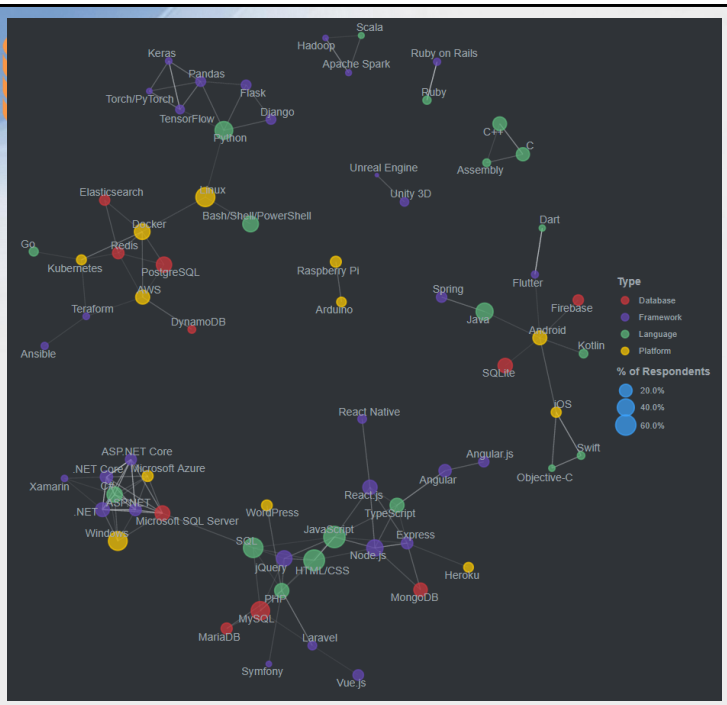
Eye opener!?
DBMS (and other fields of interest) 2023

Professional Developers | Learning to Code | Other Coders

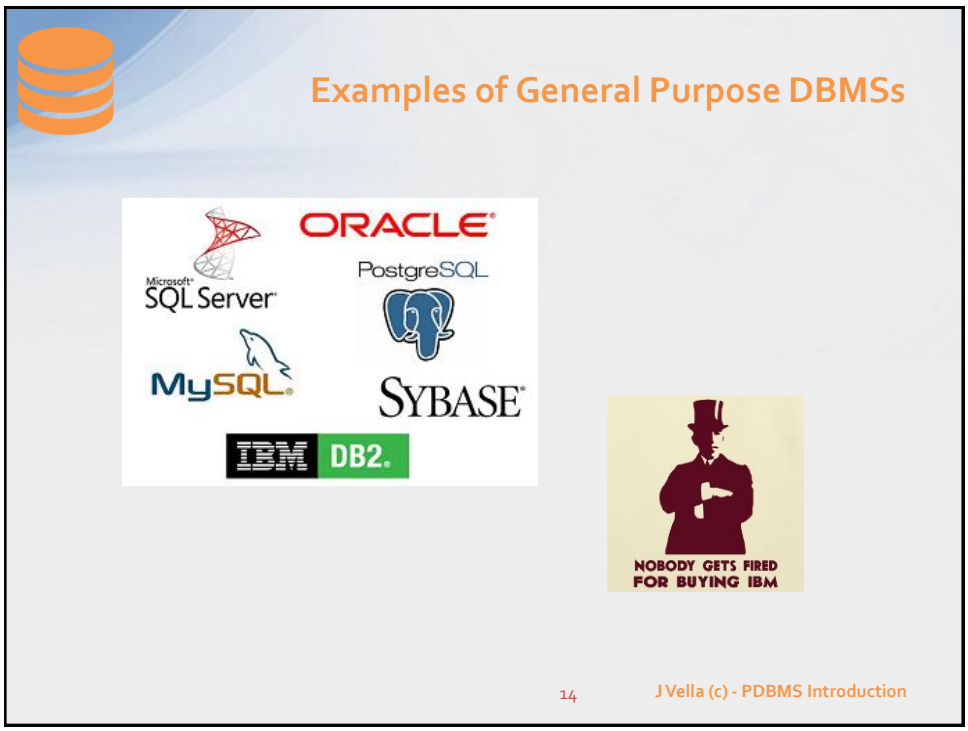
55,540 responses



12


Technology Affinity
Scala
Hadoop
Keras
Apache Spark
Ruby on Rails
Pandas
Ruby
Flask
Torch/PyTorch
Django
TensorFlow
Python
C++
C
Assembly
Unreal Engine
Unity 3D
Elasticsearch
Linux
Bash/Shell/PowerShell
Docker
Go
Redis
Kubernetes
PostgreSQL
AWS
Raspberry Pi
Arduino
Teraform
DynamoDB
Ansible
Dart
Flutter
Spring
Firebase
Java
Android
Kotlin
SQLite
Type
Database
Framework
Language
Platform
% of Respondents
20.0%
40.0%
60.0%
React Native
iOS
Swift
Angular.js
Angular
Objective-C
ASP.NET Core
.NET Core / Microsoft Azure
Xamarin
C#
ASP.NET
.NET
Microsoft SQL Server
Windows
WordPress
React.js
TypeScript
JavaScript
Express
SQL
Node.js
jQuery
HTML/CSS
Heroku
MongoDB
PHP
MySQL
MariaDB
Laravel
Symfony
Vue.js
Stackoverflow Developer Survey 2020
PDBMS Introduction

13

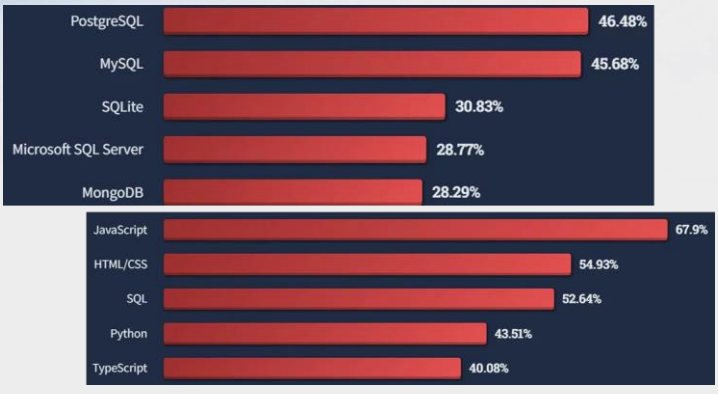

Examples of General Purpose DBMSs
ORACLE
Microsoft SQL Server
PostgreSQL
MySQL
SYBASE
IBM DB2
NOBODY GETS FIRED FOR BUYING IBM
14
J Vella (c) - PDBMS Introduction

14

Obsolete Stats from previous years

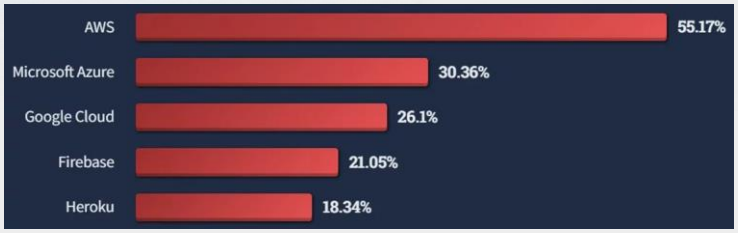
15

Eye opener!?
Databases 2022



16

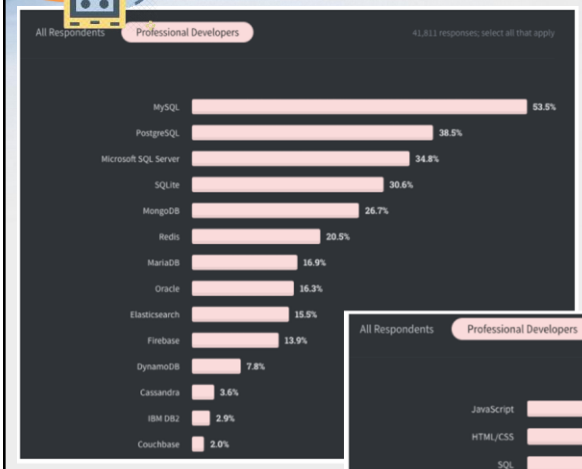
Cloud cover!? 2022

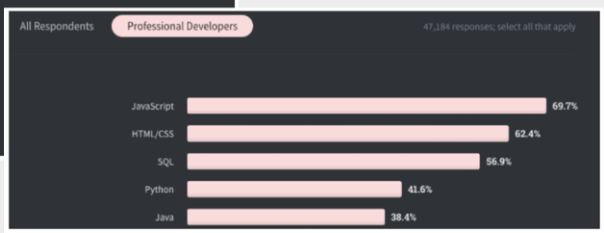


https://cloud7.news/development/stack-overflow-published-its-developer-survey-2022-results/

17

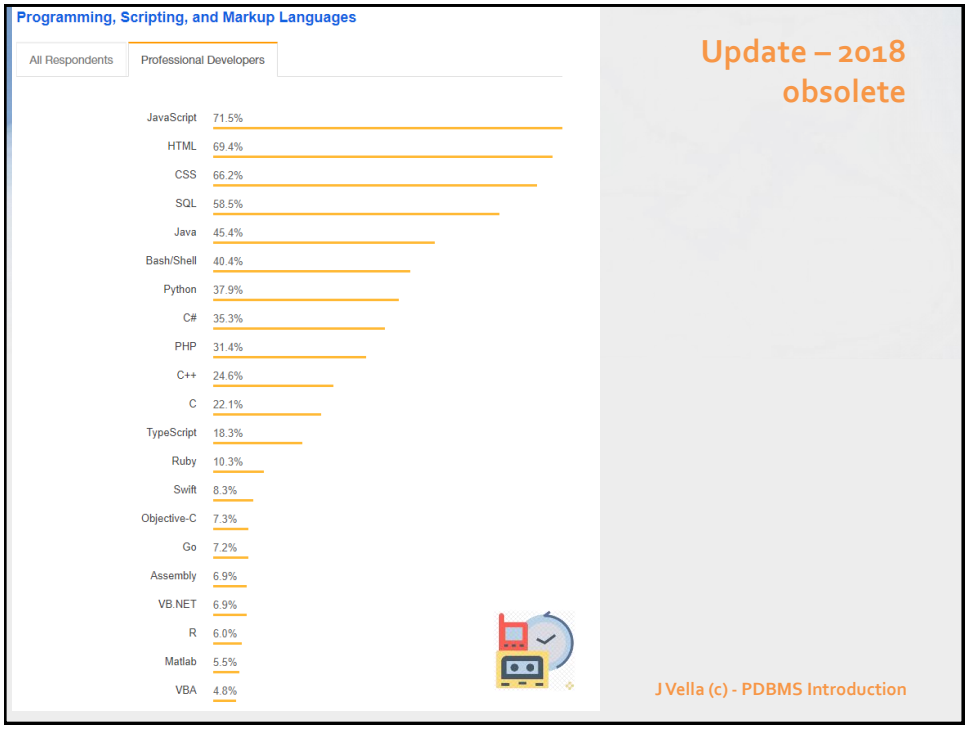
Obsolete 2020 - Eye opener!? Databases



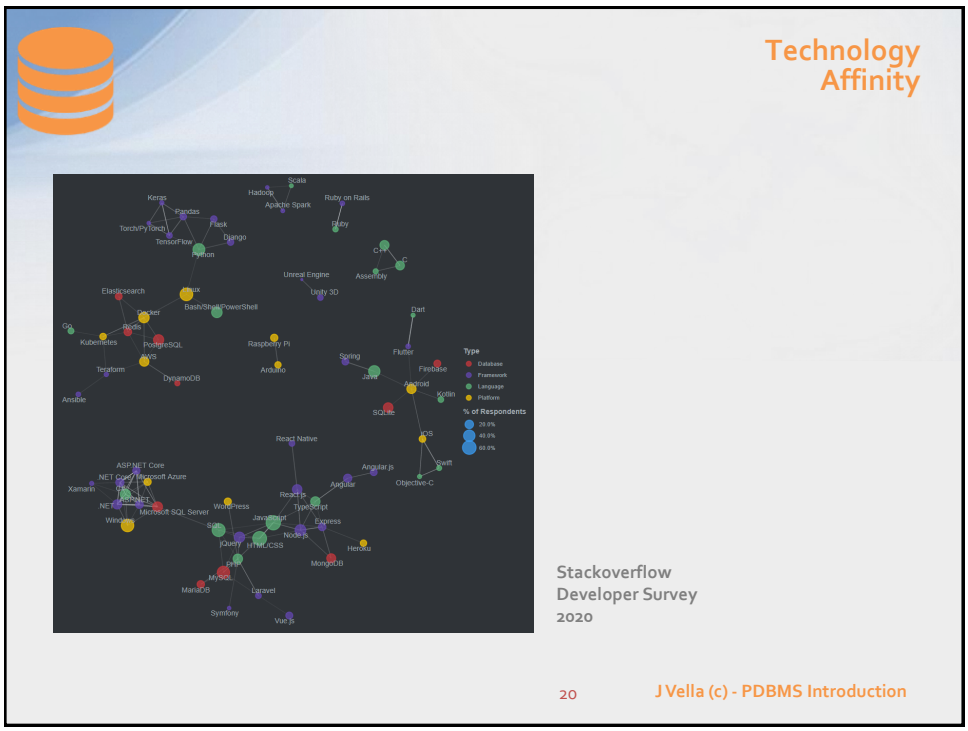


- **Look out … this numbers are personal.**
  - I.e. Not reflecting data requirements.

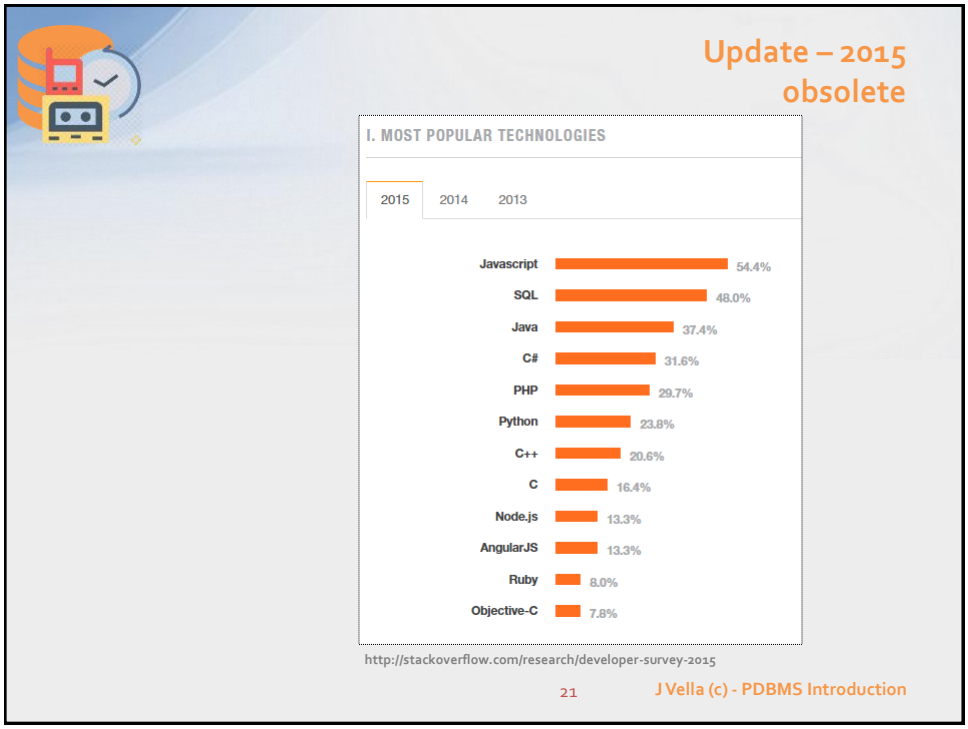
18


Programming, Scripting, and Markup Languages
All Respondents
Professional Developers
JavaScript 71.5%
HTML 69.4%
CSS 66.2%
SQL 58.5%
Java 45.4%
Bash/Shell 40.4%
Python 37.9%
C# 35.3%
PHP 31.4%
C++ 24.6%
C 22.1%
TypeScript 18.3%
Ruby 10.3%
Swift 8.3%
Objective-C 7.3%
Go 7.2%
Assembly 6.9%
VB.NET 6.9%
R 6.0%
Matlab 5.5%
VBA 4.8%
Update – 2018
obsolete
J Vella (c) - PDBMS Introduction

19


Technology
Affinity
Stackoverflow
Developer Survey
2020
20
J Vella (c) - PDBMS Introduction

20

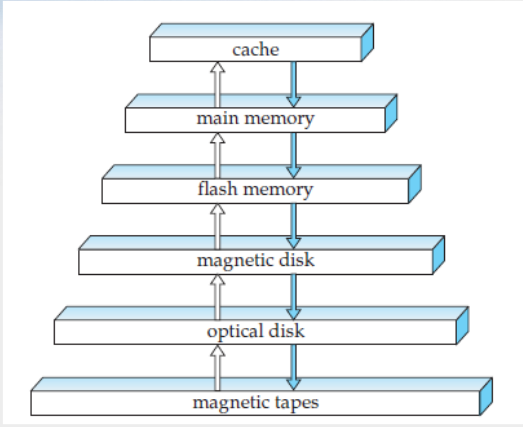

Update – 2015
obsolete
I. MOST POPULAR TECHNOLOGIES
2015
2014
2013
Javascript 54.4%
SQL 48.0%
Java 37.4%
C# 31.6%
PHP 29.7%
Python 23.8%
C++ 20.6%
C 16.4%
Node.js 13.3%
AngularJS 13.3%
Ruby 8.0%
Objective-C 7.8%
http://stackoverflow.com/research/developer-survey-2015
21
J Vella (c) - PDBMS Introduction

21


Back to Practical Aspects

22

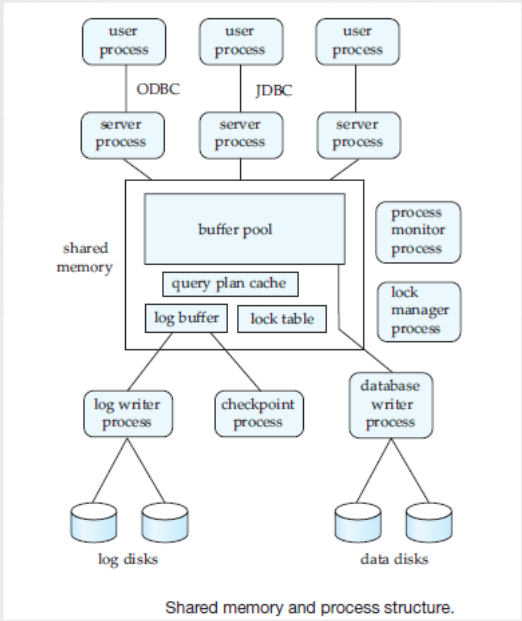
Which Resources does the DBMS Use & Manage?



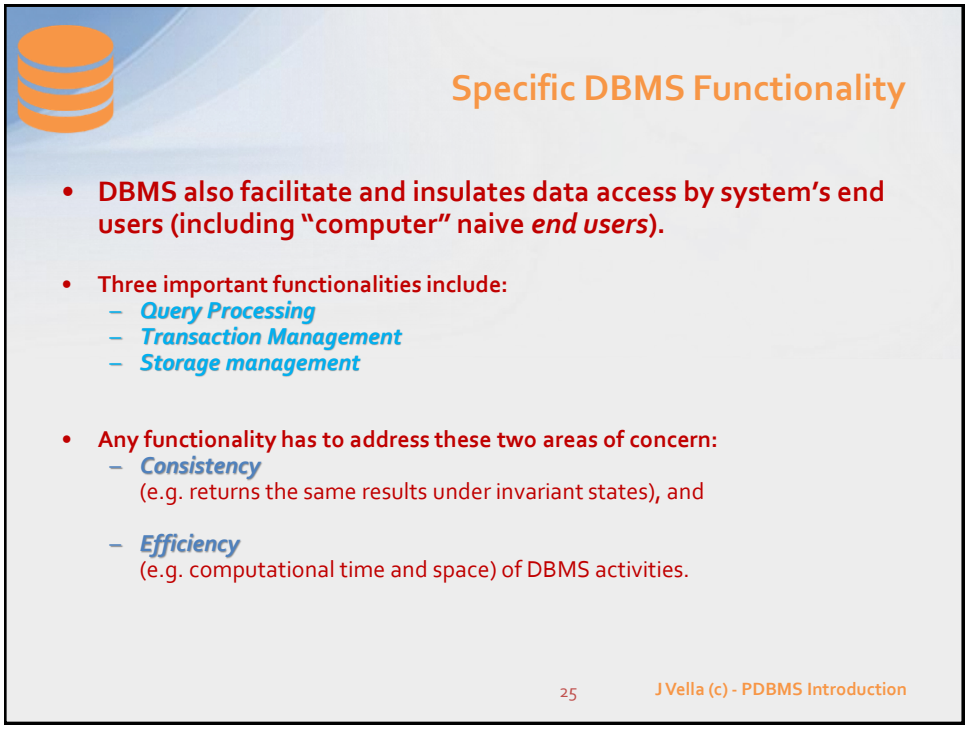
- **Each storage device differs in:**
  - Capacity;
  - Mode of access;
  - Speed & bandwidth;
  - Reliability;
  - Volatile;
  - *Cost*!

- **Please note that diagram ignores networking infrastructure related and CPUs in resources graph.**

Generic DBMS Architecture



Shared memory and process structure.


Specific DBMS Functionality
• DBMS also facilitate and insulates data access by system's end users (including "computer" naive *end users*).
• Three important functionalities include:
– *Query Processing*
– *Transaction Management*
– *Storage management*
• Any functionality has to address these two areas of concern:
– *Consistency*
(e.g. returns the same results under invariant states), and
– *Efficiency*
(e.g. computational time and space) of DBMS activities.
25
J Vella (c) - PDBMS Introduction

25


Storage Management

26

Data File per Table Structure

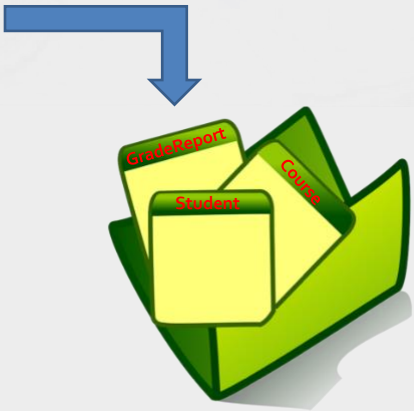
STUDENT	Name	StudentNumber	Class	Major
	Smith	17	1	CS
	Brown	8	2	CS

COURSE	CourseName	CourseNumber	CreditHours	Department
	Intro to Computer Science	CS1310	4	CS
	Data Structures	CS3320	4	CS
	Discrete Mathematics	MATH2410	3	MATH
	Database	CS3380	3	CS

SECTION	SectionIdentifier	CourseNumber	Semester	Year	Instructor
	85	MATH2410	Fall	98	King
	92	CS1310	Fall	98	Anderson
	102	CS3320	Spring	99	Knuth
	112	MATH2410	Fall	99	Chang
	119	CS1310	Fall	99	Anderson
	135	CS3380	Fall	99	Stone

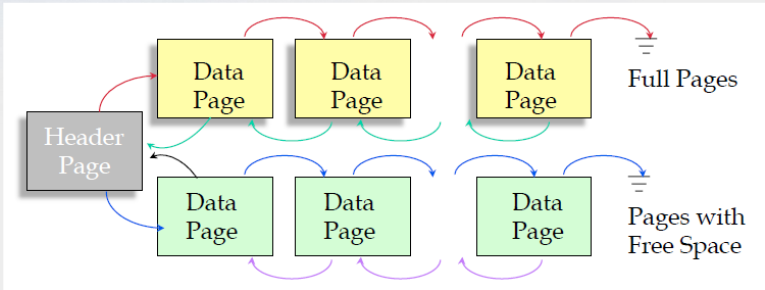
GRADE_REPORT	StudentNumber	SectionIdentifier	Grade
	17	112	B
	17	119	C
	8	85	A
	8	92	A
	8	102	B
	8	135	A

PREREQUISITE	CourseNumber	PrerequisiteNumber
	CS3380	CS3320
	CS3380	MATH2410
	CS3320	CS1310



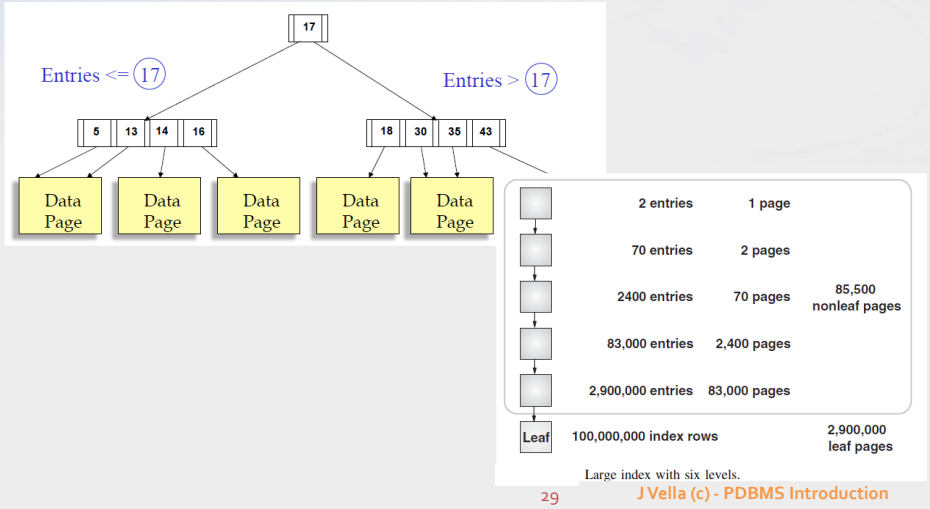
27

Heap Data Structure



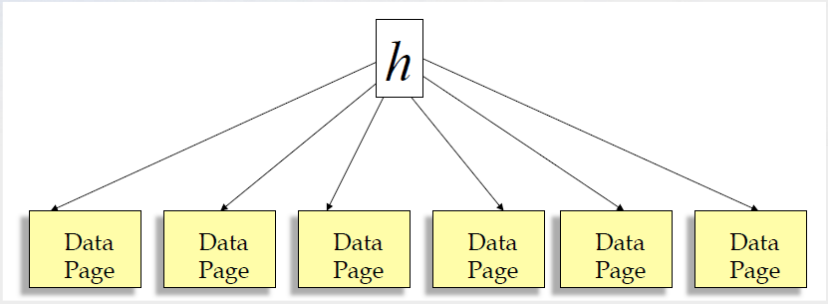
28

Basic (tree type) Indexing



29

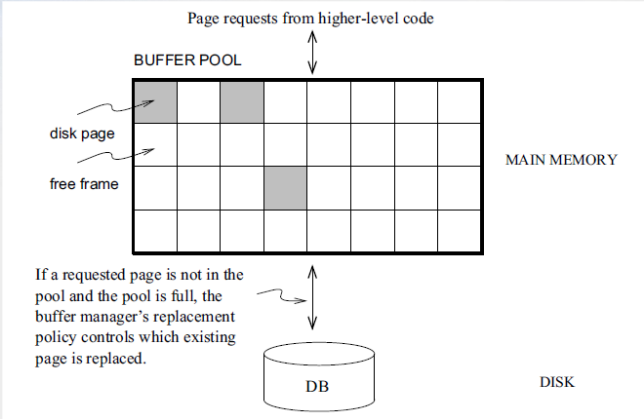
Basic (hash based) Indexing



For key based access this is as close as greatness as one can get!?

30

DBMS Main Memory Buffer - aka *Cache*

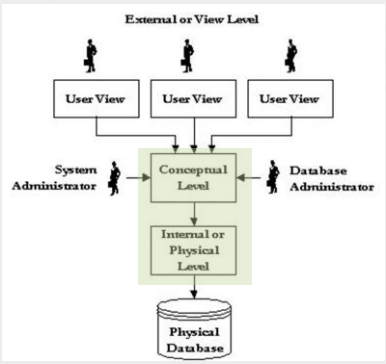


There are some DBMS that structure their data files (DB in diagram) as a paged heap! *mmap* for Unix/Linux systems.

31

Storage Manager is really doing a lot of work!

- **Many options for Physical Layer implementation of a Conceptual Schema;**
  - A high price is being paid to accommodate availability and consistency.
  - Choose fit for purpose.
    - **Beware some requirements are contradictory.**



32

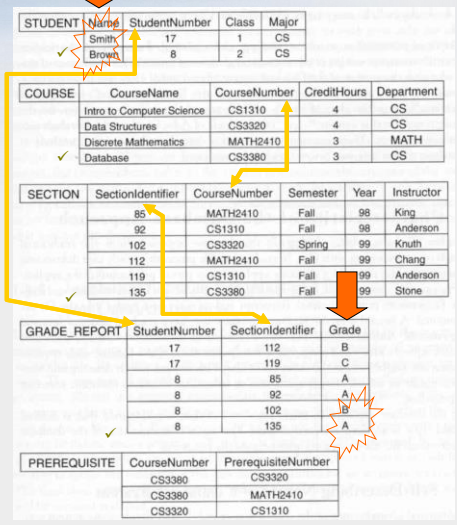
33

Query Processing & Optimisation

- We now briefly describe the structure and functionality of a *query processor* -- a process that transforms and executes a user-defined query against a database state and returns those instances that satisfy the query.
- A *declarative query* is a data access expression that indicates the structure and the property of the result (e.g. what you want). In contrast to a procedural query that specifies structure, property and an exact computational program(e.g. how to get).
  - Efficiency of a ***declarative relational query processor*** (for languages such as **SQL**) is a major project.
    - *Query optimisation* is the tuning of queries and storage structures to favour the performance of *some* frequently used query.
- What is a data access path?
  - A procedure to traverse and access required data items through the use of available look-up methods associated to the data file organisation.

34

QP Example:
Return all student's name and grade in the database unit.



STUDENT	Name	StudentNumber	Class	Major
	Smith	17	1	CS
✓	Brown	8	2	CS

COURSE	CourseName	CourseNumber	CreditHours	Department
	Intro to Computer Science	CS1310	4	CS
	Data Structures	CS3320	4	CS
	Discrete Mathematics	MATH2410	3	MATH
✓	Database	CS3380	3	CS

SECTION	SectionIdentifier	CourseNumber	Semester	Year	Instructor
	85	MATH2410	Fall	98	King
	92	CS1310	Fall	98	Anderson
	102	CS3320	Spring	99	Knuth
	112	MATH2410	Fall	99	Chang
	119	CS1310	Fall	99	Anderson
	135	CS3380	Fall	99	Stone

GRADE_REPORT	StudentNumber	SectionIdentifier	Grade
	17	112	B
	17	119	C
	8	85	A
	8	92	A
	8	102	B
✓	8	135	A

PREREQUISITE	CourseNumber	PrerequisiteNumber
	CS3380	CS3320
	CS3380	MATH2410
	CS3320	CS1310

Step One: Identify the attributes to output
i.e. Student / Name & Grade_report / Grade.

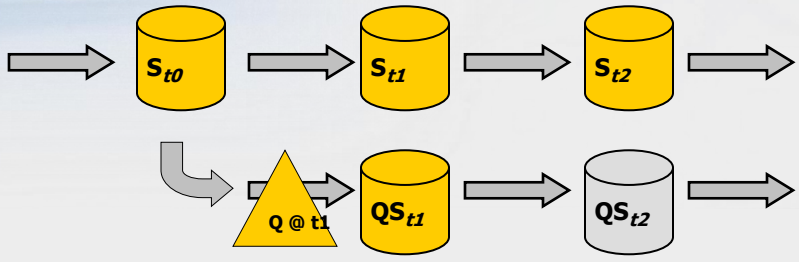
Step Two: Identify the relationships between structures of interest to the query
i.e. a) Student / StudentNumber to Grade_report / StudentNumber,
b) Course / CourseNumber to Section / CourseNumber,
c) Section / SectionIdentifier to Grade_report / SectionIdentifier.

Step Three: Apply any restrictions
i.e. Course / Coursename is equal to 'Database'.

Step Four: Compute the output.
i.e. Student 'Brown' with a grade of 'A'.

35

QP while the database is available (to other users)



At time t1, query Q's state (i.e. result QS) is related to the database state at t0.

Database states at t0 and t1 are equivalent (i.e. St0 = St1).

Database states at t1 and t2 are not necessary equivalent (i.e. St2 <> St1).

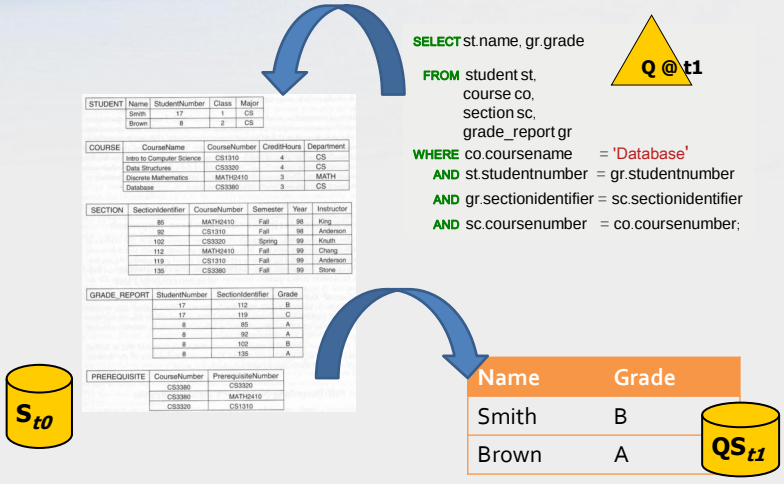
At time t2, query Q's state (i.e. result QS at t2) is NOT related to the database sate at t2.

Query Q states at t1 and t2 are equivalent (i.e. QSt1 = QSt2).

The end!

36

QP Example – another view



37

Relational Algebra

- **Fundamental operators are:**
  - Unary **SELECT** (δ) – as in restriction with a predicate
  - Unary **PROJECT** (Π) – a list of attributes
  - Binary **UNION** ($\cup$) – of two relations
  - Binary **SET DIFFERENCE** (-) – of two relations
  - Binary **PRODUCT** ($\times$) – of two relations
- **These are *Code's complete set* and have a special pitch in database theory.**
  - Nonetheless the expression built with these are not equivalent to a computer language!
  - Also the **SPJ** (select, project and product) combinations of expression attracted much interest from database theorist when describing query processing and optimisation.
- **Other operators are defined through the above five:**
  - JOIN ($\bowtie$), INTERSECT, DIVISION, etc.

38

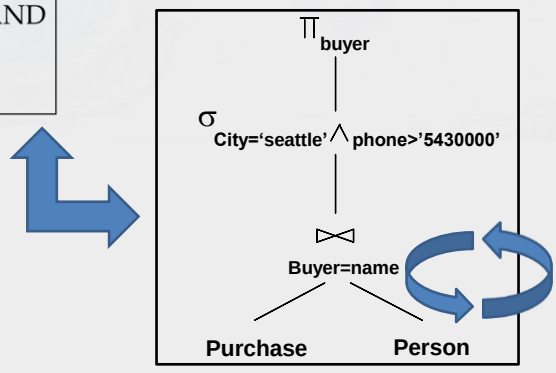
Relational Algebra (cont.)

- **Each RA operator has its own mathematical properties:**
  - Commutativity;
  - Associativity;
  - Distributive;
  - (Non) Monotonicity.
- **These properties, and the nature of the operators, allow us to express a query construct with a multiple of mathematically equivalent expressions!**
  - Different expressions that are equivalent yield the same output but arrive at the result in a different manner (e.g. program).
  - This is most useful.
- **RA needs to be augmented with a number of other operators to express, for example, most of SQL's SELECT statement!**
  - Truth be told we need a major revamp to the type system as SQL is bag type not set type.
- **RA is useful in QP & QO of declarative languages. Also applicable for parallel implementations.**
  - Many BigData languages have a basis in extended relational algebra languages.

39

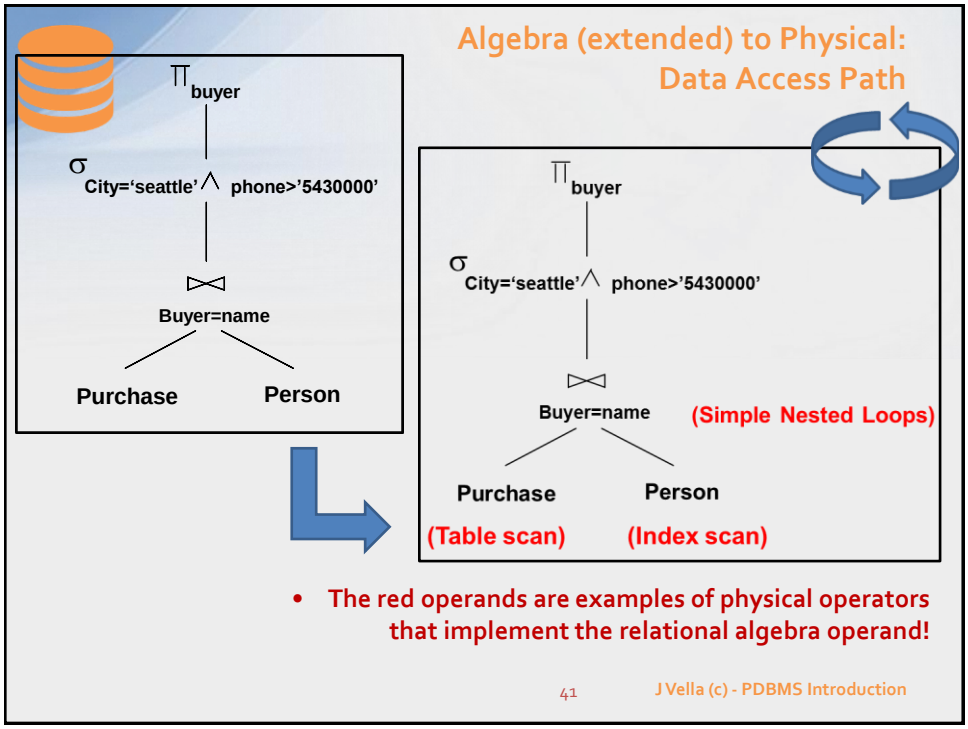
Convert an SQL query into Relational Algebra

```
SELECT S.buyer
FROM Purchase P, Person Q
WHERE P.buyer=Q.name AND
      Q.city='seattle' AND
      Q.phone > '5430000'
```

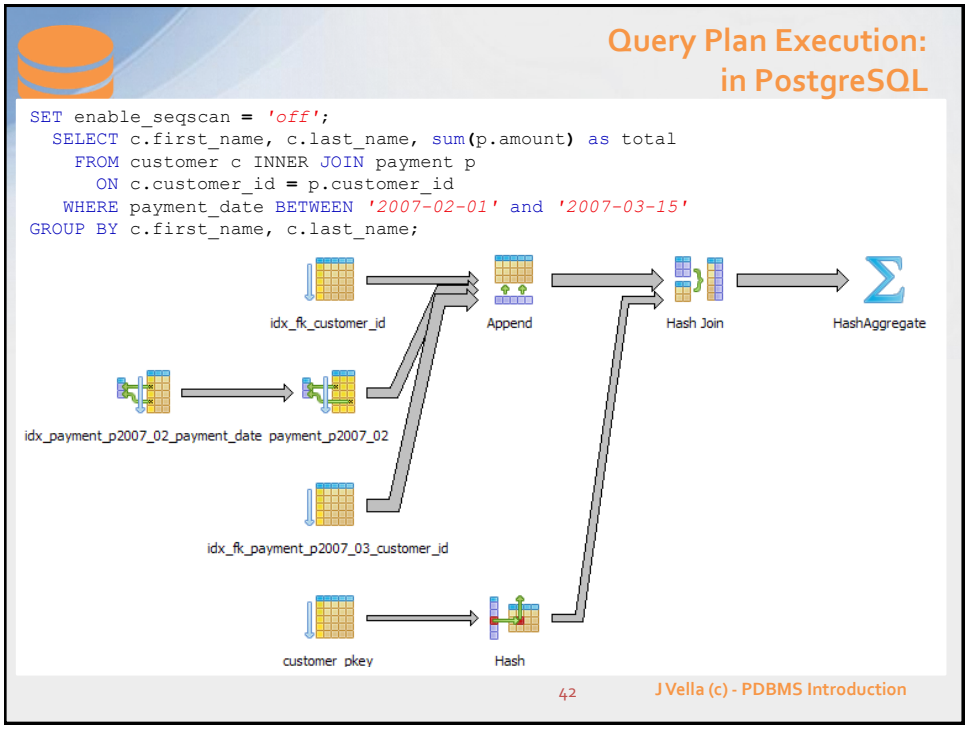


Declarative SQL query ⟶ *Imperative query execution plan:*

40

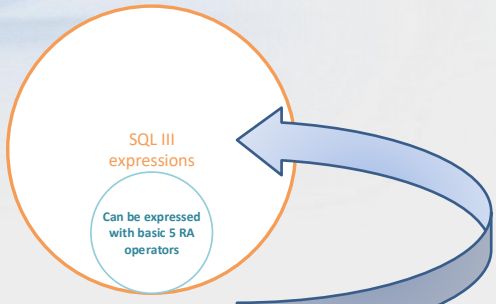

Algebra (extended) to Physical:
Data Access Path
buyer
City='seattle' ∧ phone>'5430000'
Buyer=name
Purchase
Person
buyer
City='seattle' ∧ phone>'5430000'
Buyer=name
(Simple Nested Loops)
Purchase
Person
(Table scan)
(Index scan)
• The red operands are examples of physical operators that implement the relational algebra operand!
41
J Vella (c) - PDBMS Introduction

41


Query Plan Execution:
in PostgreSQL
SET enable_seqscan = 'off';
SELECT c.first_name, c.last_name, sum(p.amount) as total
FROM customer c INNER JOIN payment p
ON c.customer_id = p.customer_id
WHERE payment_date BETWEEN '2007-02-01' and '2007-03-15'
GROUP BY c.first_name, c.last_name;
idx_fk_customer_id
Append
Hash Join
HashAggregate
idx_payment_p2007_02_payment_date
payment_p2007_02
idx_fk_payment_p2007_03_customer_id
customer_pkey
Hash
42
J Vella (c) - PDBMS Introduction

42

Additions to Relational Algebra



- **Group By operator**
- **Partition/Frame operator**
  - (not data partitioning)
- **Map**
- **Sort**
- **Transitive closure, Fixed Point (Recursion)**

43

Procedural operator for Group By queries

```
-- q1
SELECT job, sal
FROM scott.emp
ORDER BY job, sal;
```

Q1

ANALYST	3000
ANALYST	3000
CLERK	800
CLERK	950
CLERK	1100
CLERK	1300
MANAGER	2450
MANAGER	2850
MANAGER	2975
PRESIDENT	5000
SALESMAN	1250
SALESMAN	1250
SALESMAN	1500
SALESMAN	1600

```
-- q2
SELECT job, sal
FROM scott.emp
WHERE job <> 'PRESIDENT'
ORDER BY job;
```

Q2

ANALYST	3000
ANALYST	3000
CLERK	800
CLERK	1100
CLERK	950
CLERK	1300
MANAGER	2975
MANAGER	2450
MANAGER	2850
SALESMAN	1250
SALESMAN	1600
SALESMAN	1250
SALESMAN	1500

```
-- q3
SELECT job, sum(sal)
FROM scott.emp
WHERE job<>'PRESIDENT'
GROUP BY job
ORDER BY job;
```

Q3

ANALYST	6000
CLERK	4150
MANAGER	8275
SALESMAN	5600

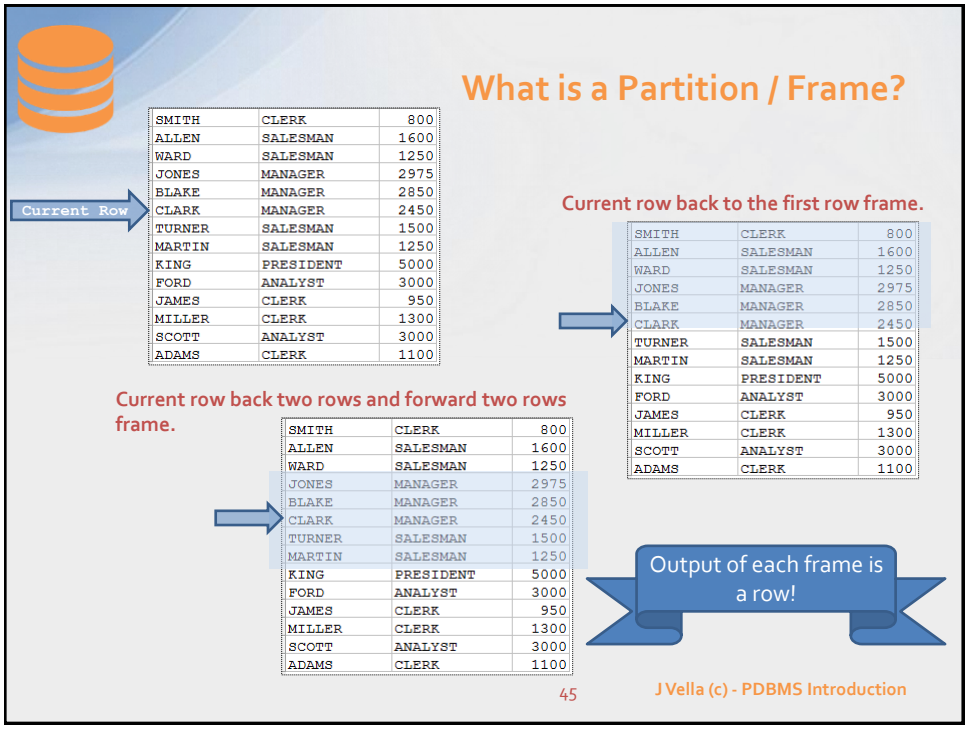
```
-- q4
SELECT job, sum(sal)
FROM scott.emp
WHERE job<>'PRESIDENT'
GROUP BY job
HAVING sum(sal) > 5750
ORDER BY job;
```

Q4

ANALYST	6000
MANAGER	8275

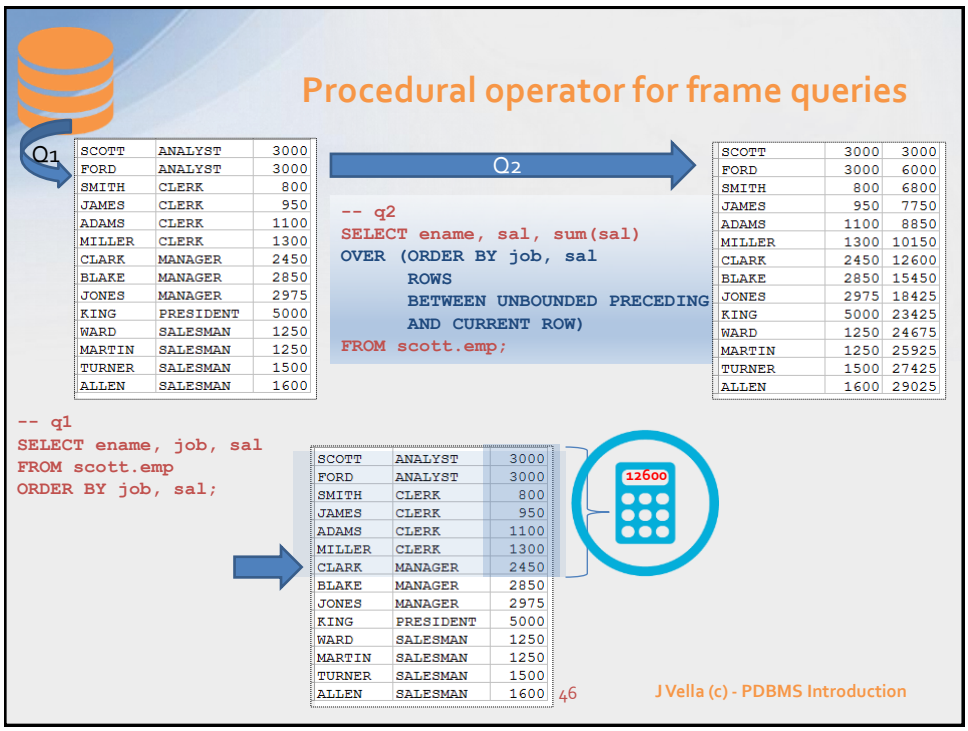
Output of each band is at most a row!

44



SMITH	CLERK	800
ALLEN	SALESMAN	1600
WARD	SALESMAN	1250
JONES	MANAGER	2975
BLAKE	MANAGER	2850
CLARK	MANAGER	2450
TURNER	SALESMAN	1500
MARTIN	SALESMAN	1250
KING	PRESIDENT	5000
FORD	ANALYST	3000
JAMES	CLERK	950
MILLER	CLERK	1300
SCOTT	ANALYST	3000
ADAMS	CLERK	1100

45



SCOTT	ANALYST	3000
FORD	ANALYST	3000
SMITH	CLERK	800
JAMES	CLERK	950
ADAMS	CLERK	1100
MILLER	CLERK	1300
CLARK	MANAGER	2450
BLAKE	MANAGER	2850
JONES	MANAGER	2975
KING	PRESIDENT	5000
WARD	SALESMAN	1250
MARTIN	SALESMAN	1250
TURNER	SALESMAN	1500
ALLEN	SALESMAN	1600

SCOTT	3000	3000
FORD	3000	6000
SMITH	800	6800
JAMES	950	7750
ADAMS	1100	8850
MILLER	1300	10150
CLARK	2450	12600
BLAKE	2850	15450
JONES	2975	18425
KING	5000	23425
WARD	1250	24675
MARTIN	1250	25925
TURNER	1500	27425
ALLEN	1600	29025

46

Procedural operator for map queries

```
-- q1
SELECT ename, hiredate
FROM scott.emp
WHERE job='CLERK'
ORDER BY ename;
```

Q1

ename character varying(10)	hiredate date
ADAMS	1983-01-12
JAMES	1981-12-03
MILLER	1982-01-23
SMITH	1980-12-17

Q2

```
-- q2
SELECT ename,
       scott.date2julian(hiredate)
       as "hiredate"
FROM scott.emp
WHERE job='CLERK'
ORDER BY ename;
```

ename character varying(10)	hiredate bigint
ADAMS	2445347
JAMES	2444942
MILLER	2444993
SMITH	2444591

```
-- function definition
-- Notice its datatypes;
-- input type is date
-- output type is bigint

CREATE OR REPLACE FUNCTION
       scott.date2julian(date)
RETURNS bigint
LANGUAGE plpgsql
AS $function$
BEGIN
    RETURN TO_CHAR($1, 'J');
END; $function$;

SELECT scott.date2julian(
     now()::date) AS "TODAY";
```

Use of function

TODAY bigint
2457417

Side Note:

- **Julian day number** is an integer representing solar days since January 1st 4713 BC.

47

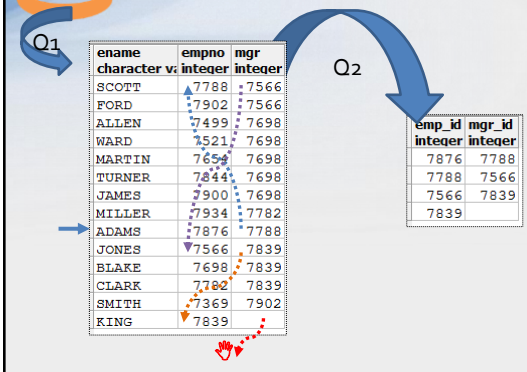
Procedural operator for Transitive closure queries (i)

```
-- q1
WITH RECURSIVE factorial(n, fact)
AS ( SELECT 1::BIGINT, 1::BIGINT
     UNION ALL
     SELECT n+1, (n+1)*fact
     FROM factorial
     WHERE n+1<15 )
 SELECT * FROM factorial;
```

	n bigint	fact bigint
1	1	1
2	2	2
3	3	6
4	4	24
5	5	120
6	6	720
7	7	5040
8	8	40320
9	9	362880
10	10	3628800
11	11	39916800
12	12	479001600
13	13	6227020800
14	14	87178291200

48

Procedural operator for Transitive closure queries (ii)



ename character v	empno integer	mgr integer
SCOTT	7788	7566
FORD	7902	7566
ALLEN	7499	7698
WARD	7521	7698
MARTIN	7654	7698
TURNER	7844	7698
JAMES	7900	7698
MILLER	7934	7782
ADAMS	7876	7788
JONES	7566	7839
BLAKE	7698	7839
CLARK	7782	7839
SMITH	7369	7902
KING	7839	

emp_id integer	mgr_id integer
7876	7788
7788	7566
7566	7839
7839	

```
-- q2
WITH RECURSIVE eh(emp_id,mgr_id) AS
(  SELECT e.empno, e.mgr
     FROM scott.emp e
    WHERE e.empno = 7876
UNION ALL
   SELECT m.empno, m.mgr
     FROM scott.emp m, eh
    WHERE m.empno=eh.mgr_id
 )
 SELECT * FROM eh;
```

```
-- q1
SELECT ename, hiredate
FROM scott.emp
WHERE job='CLERK'
ORDER BY ename;
```


49

So Why Query Optimization?

- Who is on the receiving end?
- What does it entail in a software development cycle?

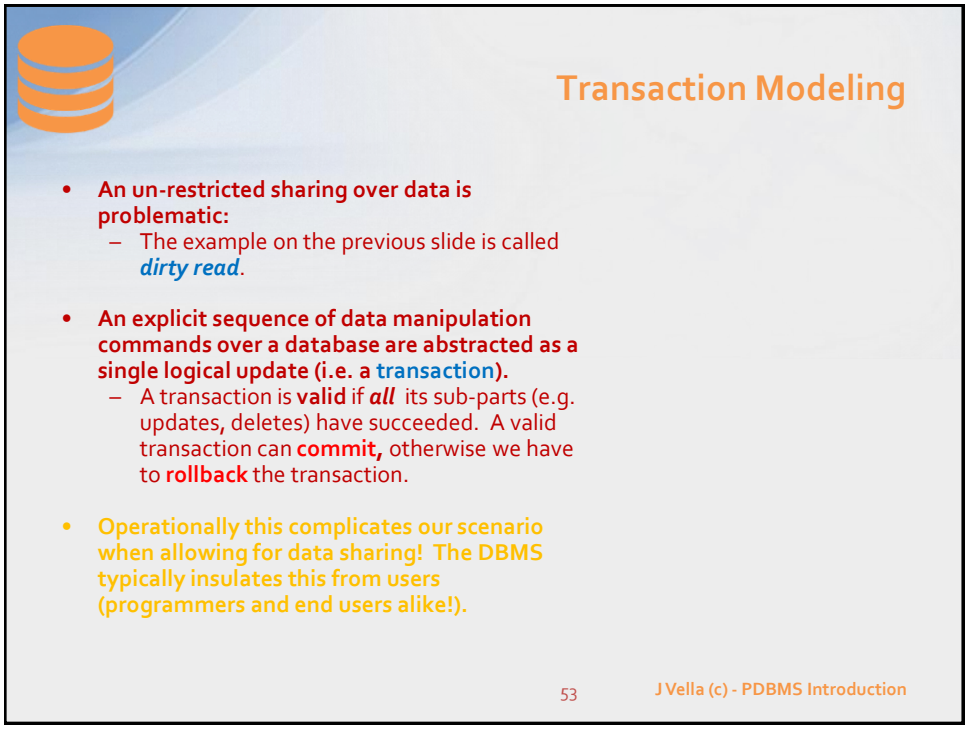
50

51

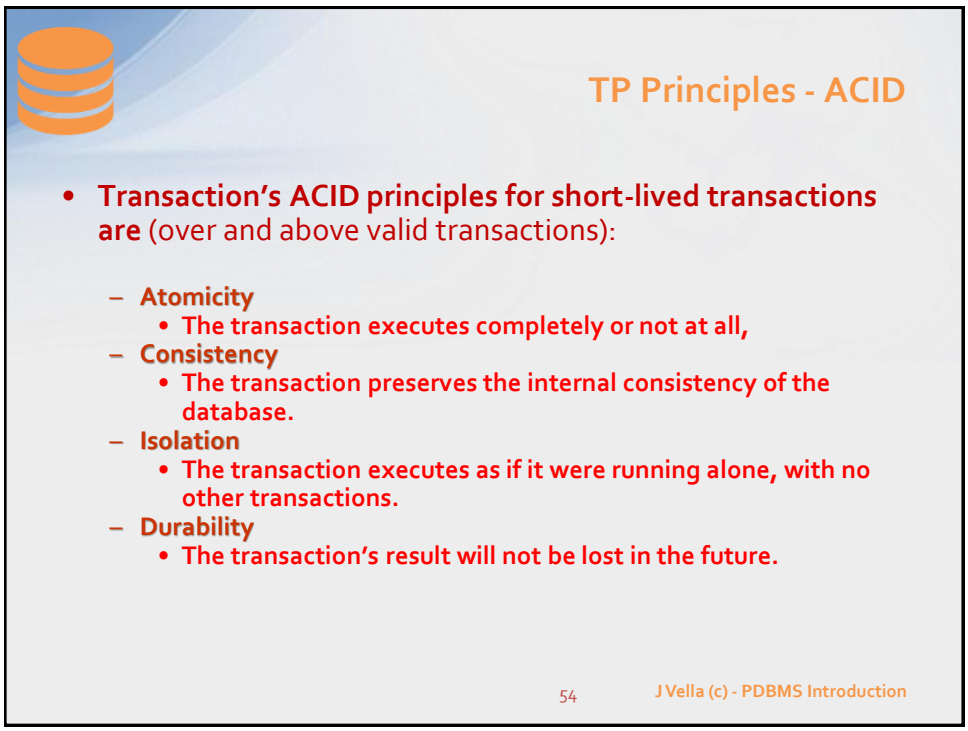
Unrestricted access to a common database state: dirty read

Time	Database state	Process & Trans. 1 Transaction	Process & Trans. 1 Local memory	Process and Trans. 2 Transaction	Process and Trans. 2 Local memory	Comments
			X=0		Y=0	
1	A=10, & B=20	read(A,X)	X=10			Read db rec A to var X.
2	A=10, & B=20	X:=X+100	X=110			
3	A=10, & B=20	write(A,X)	X=110			Write var X to db A.
4	A=110, & B=20			read(A,Y)	Y=110	T2 read from T1, and T1 is not yet closed.
5	A=110, & B=20			write(B,Y)	Y=110	
6	A=110, & B=110	Rollback	x=110			
7	A=10, & B=110					At this point B is incorrect

52


Transaction Modeling
• An un-restricted sharing over data is problematic:
– The example on the previous slide is called dirty read.
• An explicit sequence of data manipulation commands over a database are abstracted as a single logical update (i.e. a transaction).
– A transaction is valid if all its sub-parts (e.g. updates, deletes) have succeeded. A valid transaction can commit, otherwise we have to rollback the transaction.
• Operationally this complicates our scenario when allowing for data sharing! The DBMS typically insulates this from users (programmers and end users alike!).
53
J Vella (c) - PDBMS Introduction

53


TP Principles - ACID
• Transaction's ACID principles for short-lived transactions are (over and above valid transactions):
– Atomicity
• The transaction executes completely or not at all,
– Consistency
• The transaction preserves the internal consistency of the database.
– Isolation
• The transaction executes as if it were running alone, with no other transactions.
– Durability
• The transaction's result will not be lost in the future.
54
J Vella (c) - PDBMS Introduction

54

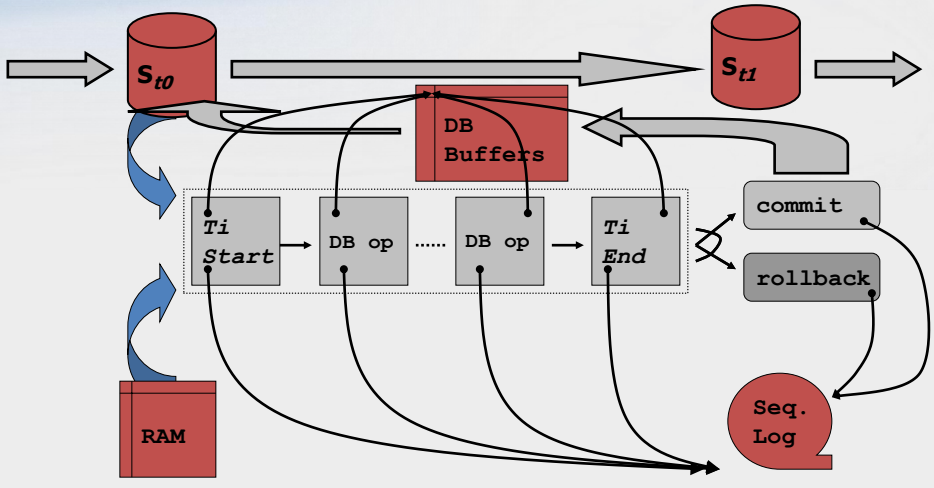
Transaction Modeling: an Example

- "Ensure that the employees salary is upped by 25% and each salesperson commission is augmented by Euro 250."

- Remark SQL command sequence

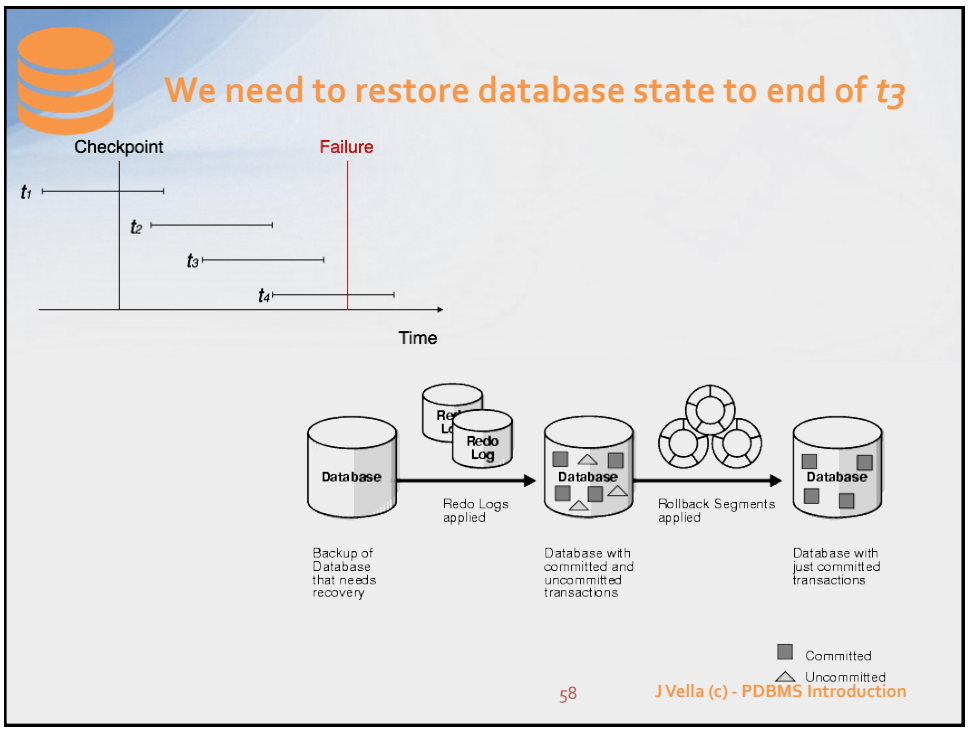
```
Remark set session
SET TRANSACTION SERIALIZABLE;
...
START transaction;
UPDATE emp
      SET sal = sal * 1.25;
Remark IMPLICIT mechanism - ON error ROLLBACK
UPDATE emp
      SET comm = comm + 250
WHERE comm IS NOT NULL;
Remark IMPLICIT mechanism - ON error ROLLBACK
COMMIT;
END transaction;
```


Transaction Modeling and Database States




Data
Recovery

57


We need to restore database state to end of t3
Checkpoint
Failure
t1
t2
t3
t4
Time
Database
Redo Log
Redo Logs applied
Database
Rollback Segments applied
Database
Backup of Database that needs recovery
Database with committed and uncommitted transactions
Database with just committed transactions
Committed
Uncommitted
58
J Vella (c) - PDBMS Introduction

58

59

These are current crop of new DBMS tool sets

- **They are great in for example:**
  - Caching;
  - Load balancing;
  - Streaming or queuing semantics work;
  - Distributing data and code (e.g. map-reduce).
- **But *caveat emptor* ("Let the buyer beware"):**
  - These are not general-purpose DBMS:
    - **Check if ACID or CUASAL or BASE transaction processing;**
    - **Check data modelling capabilities (even compare with RDB);**
    - **What about its Query Language – e.g. is it declarative?**
    - **Is data security mechanism more than an "all or nothing" access?**
- **We have used:**
  - CouchDB & MongoDB;
  - Noe4j;
  - Amazon things!?
  - Apache thingies too

60

Unit's Interest

- **Persistence of data**
- **High level of physical data independence**
- **Aid the development of the Computer Information System**
- **Reliability and efficiency (of each DBMS module individually and collectively)**
  - E.g., storage manager efficiency should come with query and transaction processing
  - E.g., query processing should come with transaction processing

61

Unit's constraint

- **Fit for purpose:**
  - Data modelling
  - Query modelling
  - Storage planning & allocation
  - Transaction processing

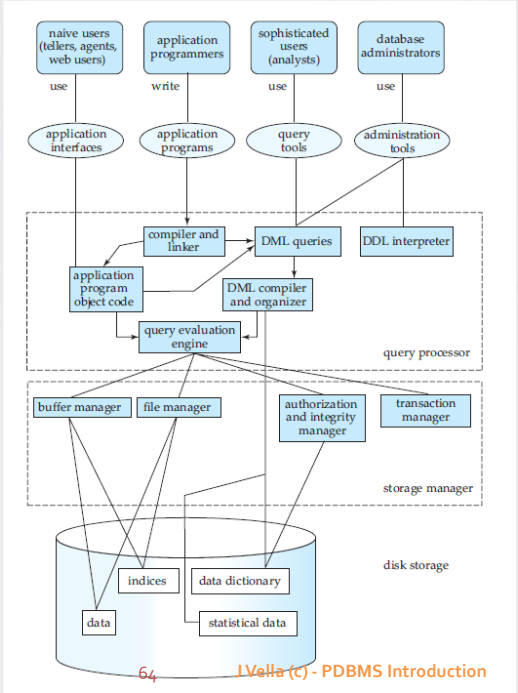
62

The Parts

(mostly revision)

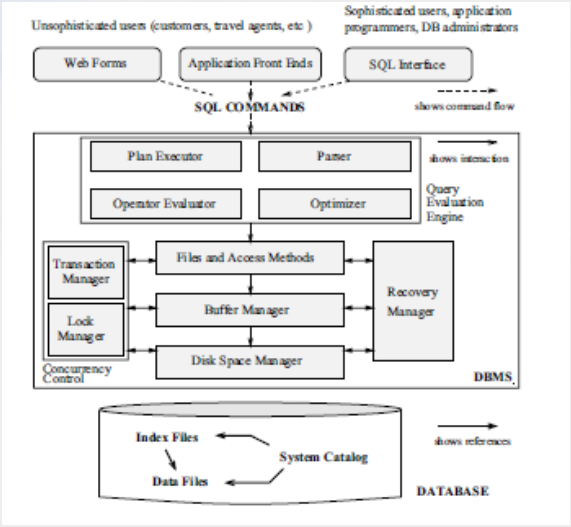
63

Generic DBMS Modules



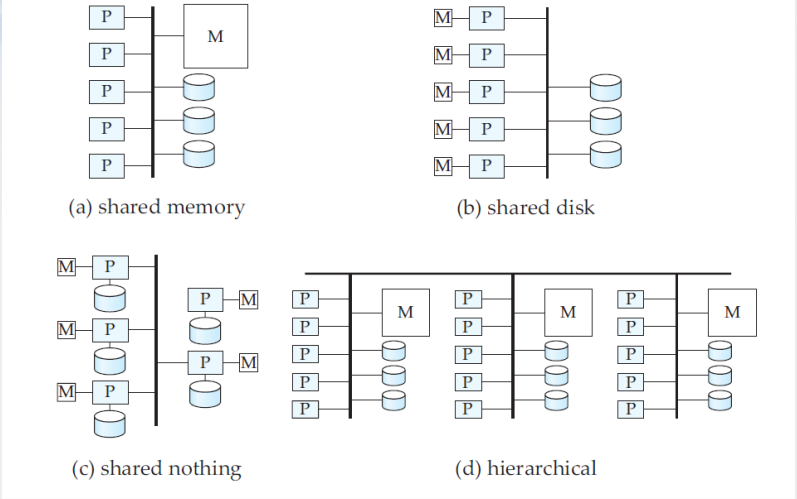
64

Generic DBMS Architecture



65

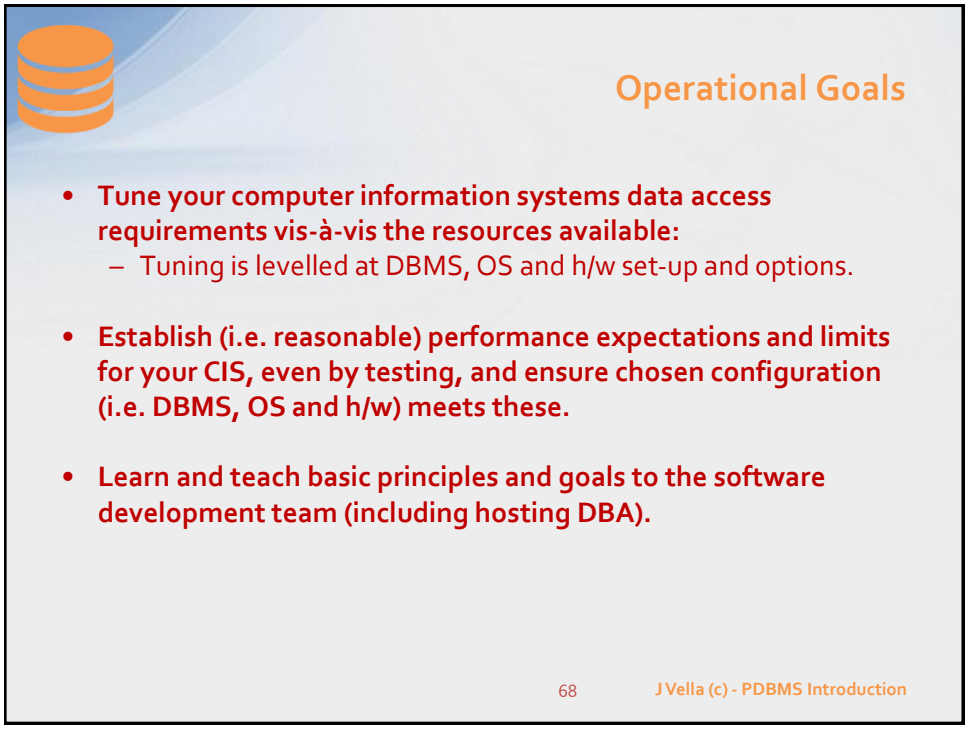
Parallel Database Architectures



66


The Key Ideas
(mostly revision)

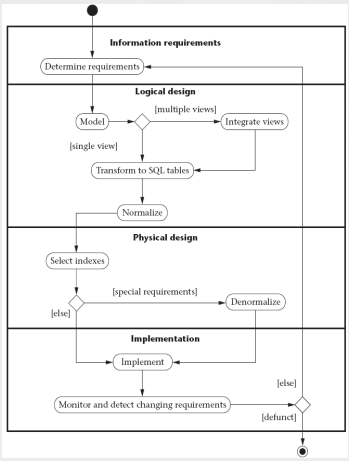
67


Operational Goals
• Tune your computer information systems data access requirements vis-à-vis the resources available:
– Tuning is levelled at DBMS, OS and h/w set-up and options.
• Establish (i.e. reasonable) performance expectations and limits for your CIS, even by testing, and ensure chosen configuration (i.e. DBMS, OS and h/w) meets these.
• Learn and teach basic principles and goals to the software development team (including hosting DBA).
68
J Vella (c) - PDBMS Introduction

68

Database life cycle (i.e. the Physical Part)

- **Once DBMS specific artifacts are introduced one can start to introduce physical objects to address *performance*, *reliability* and *connectivity* issues.**
  - Again it's best to reconcile with requirements.
  - Physical artifacts include:
    - **Variety of hardware;**
    - **Variety of indexes;**
    - **Data placement policies;**
    - **DBMS set-up parameters.**
- **Once the database is active, DBMS tools are used for monitoring performance and comparing this with expected guidelines.**



69

(mostly revision)

Guiding Principles

70

Database systems:
guiding principles (i)

ServiceCard

Serial Numb.	Service Type	Client Number	Client Name	Hours	Hrs to Charge
110	SERVICE	25010	Jeffrey	3	0
120	SERVICE	25150	Jeremey	2	2
130	REPAIR	25060	JJ	2	2
140	INSTALL	25090	Jaz	3	1
150	MA	25090	Jaz	2	0

- Consider the following data extracts.

ServiceCard

Serial Numb.	Service Type	Client Number	Hours	Hrs to Charge
110	SERVICE	25010	3	0
120	SERVICE	25150	2	2
130	REPAIR	25060	2	2
140	INSTALL	25090	3	1
150	MA	25090	2	0

ClientDetails

Client Numb.	Client Name
25010	Jeffrey
25150	Jeremey
25060	JJ
25090	Jaz

71

Database systems:
guiding principles (i) - continued

- **Consider the previous data extract:**
  - First table is not in 3nf.
  - For fixing it:
    we decompose it into the following two tables.
- **Data redundancy (i.e. at Conceptual level)**
  - Is the need to ensure, to a high degree, that data in a database is not duplicated.
    - **If duplication (i.e. redundancy) exists then inconsistency arising from it has to be addressed (e.g. extra coding, obscures data meaning).**
- **We want to minimize data redundancy at logical level.**

72

Data Independence:
Motivation & Definitions

- **The *separation* of physical data items (e.g. files), logical data items (relational database), and application programs that interact with the database minimises their inter dependence.**
  - The concept of data independence conveys our wish to make artefact changes without unduly having to *re-map* (or recompile) all the higher artefacts that depend on the artefact.
    - **Lowest is the physical level , middle is the logical and higher is the AP.**
- **Logical data independence**
  - changes to the schema do not necessitate changes in the application programs. **For example: adding new data types - programmed access should still be consistent.**
- **Physical data independence**
  - changes to the physical schema are insulated from conceptual and application programs that use the database. **Typical physical changes include: index re-structuring; records re-ordering in a data file.**

73

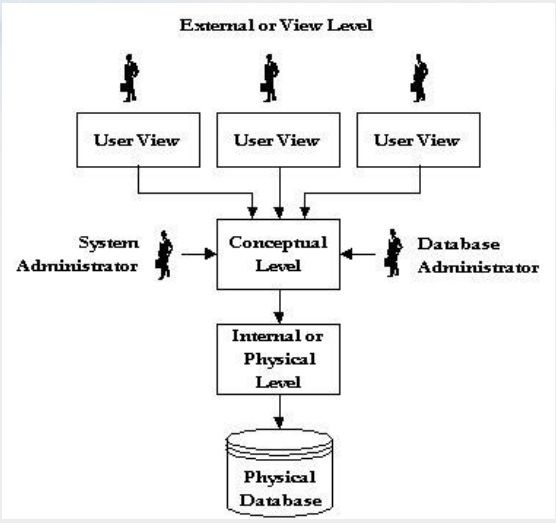
Data Independence:
ANSI-SPARC Architecture (3 level)

The classic representation of a database and DBMS data architecture is the ANSI/X3/SPARC proposal (1978). This architecture comprises three levels of abstraction.

- **Internal Schema**
  - This is the physical storage model containing information such as devices, file locations, structures, indexing systems and access methods.
- **Conceptual Schema**
  - A single and consistent model of the database of interest.
- **External Schemas**
  - A view of the conceptual model applicable users or processes.
    - **There are usually many of these and view overlapping is allowed.**

74

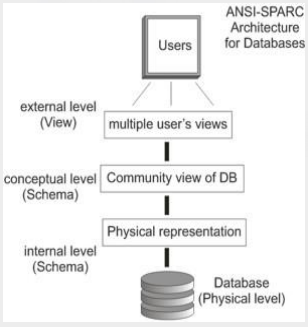
Data Independence: ANSI-SPARC Architecture (3 level)



75

Database systems: guiding principles (ii)

- **We want to minimize disruption due to schema changes.**
  - Change physical characteristics without the need to change the conceptual schema;
  - Change the external views without changing the conceptual schema;



76

Database systems:
guiding principles (iii)

- **Data security**
  - We said earlier that Data is indispensable:
    - **Protect from 'abuse' – e.g. security breaches, errors, contention, h/w failures (e.g. hard disks fail).**

"I'd just like to know how these crazy rumors get started."

"I don't believe it!" – but true!?
http://datacent.com/failing_hard_drive_sounds